FRONTEND SIMPLIFIED

JavaScript Cheat Sheet



Everything from the JavaScript lessons, with examples. HTML is the structure, CSS is the design, **JavaScript is the behavior.**

1 · Basics 2 · Logic & Loops 3 · Functions & Arrays 4 · Objects, DOM & Async

01 The 5 basic data types

Type	What it holds	Example
String	Text, wrapped in quotes	`"David" 'hello'`
Number	Any number, no quotes	`20 0.5 -3`
Boolean	A yes/no: `true` or `false`. Drives every `if`	`let loggedIn = true;`
null	"Empty," and **you** set it on purpose	`let user = null;`
undefined	Never set. JavaScript hands this back	`let score; // undefined`

Not sure what type something is? Put `typeof` in front: `typeof 20` → `"number"`

02 Strings

Any quotes work. **No quotes = JS thinks it's a variable.**

D	a	v	i	d
0	1	2	3	4

Index starts at 0
`.length` = 5
Last: `length - 1`

```
let str = "David";
str[0];              // "D"
str[str.length - 1]; // "d"
"hi, " + "David";    // join with +
`hi, ${name}`;     // backticks + ${ }
```

03 Numbers

Op	Does	Example
`+` `-`	Add, subtract	`10 - 3 → 7`
`*`	Multiply	`10 * 0.5 → 5`
`/`	Divide	`20 / 4 → 5`
`%`	**Remainder**	`15 % 5 → 0`
`( )`	Runs first	`100 / (4 * 5) → 5`

Even or odd? If `num % 2 === 0`, it's even. You'll use `%` more than you expect (hello, FizzBuzz).

04 Variables

`const`	**Can't** change. Your go-to
`let`	**Can** change later
`var`	Old. Skip it

```
let temperature = 20;
temperature = temperature + 2; // 22
const fullName = "David Bragg";
```

No spaces in names. Use **camelCase**: `fullName`, `isStoreOpen`.

05 Comparing values

`=`	**Assigns** a value. Doesn't compare!
`==`	Value only: `"1" == 1` → true
`===`	Value **and** type: `"1" === 1` → false
`!==`	Not equal: `"1" !== 1` → true

✓ **Use `===` almost every time.** `==` ignores type, and that's how quiet bugs slip in. `!` flips any check.

BONUS Not covered in the lessons yet, but good to know.

06 Conditionals: if / else if / else

```
if (subscribed) {
  console.log("Show the video");
} else if (loggedIn) {
  console.log("Upgrade your plan");
} else {
  console.log("Log into your account");
}
```

Only one branch runs: the first true one, so order matters. An `if` can stand alone; an `else` can't.

Operator	Asks
`===` `!==`	Equal? Not equal?
`>` `<`	Greater than? Less than?
`>=` `<=`	Same, but the equal case counts too
`&&`	**And:** true only if **both** sides are true
`||`	**Or:** true if **either** side is true
`!`	**Not:** flips true and false

`if (isOpen)` = `if (isOpen === true)`. `!isOpen` = closed?

Truthy and falsy

An `if` just needs a **truthy** value. These 6 are **falsy**:

`false` `0` `""` `null` `undefined` `NaN`

Everything else is truthy, even `"0"`, `"false"`, `[]` and `{}`. Unsure? `!!value` in the console tells you.

The ternary: an if/else on one line

```
hot  ? "Hot" : "Cold"
```

condition · if true · if false

Great when each outcome is short. Several lines of logic? Use an `if` block.

07 Loops: repeat until a condition says stop

DRY: Don't Repeat Yourself. Copying the same line over and over? That's the signal to reach for a loop.

```
for ( let i = 0 ;  i < 100  ;  i++  ) { ... }
```

1. Start here · 2. Keep going while true · 3. Step after each pass

FizzBuzz (the classic interview test)

```
for (let i = 1; i <= 20; i++) {
  if (i % 3 === 0 && i % 5 === 0) {
    console.log("front end simplified");
  } else if (i % 3 === 0) {
    console.log("front end");
  } else if (i % 5 === 0) {
    console.log("simplified");
  } else {
    console.log(i);
  }
}
```

The "both" case goes **first**. Write `i % 3 === 0`, not `i % 3` (0 is falsy!).

Loop over a string

```
for (let i = 0; i < str.length; i++) {
  console.log(str[i]);
}
```

Use `.length`, never a hard-coded number, so it fits any string.

The while loop

```
let count = 1;
while (count <= 100) {
  console.log(count);
  count++;
}
```

Infinite loop alert: something inside must move toward stopping. Forget `i++` and the browser freezes.

`i++`, `++i` and `i = i + 1` all add one. Start at `0` with `i < 100` = 100 passes (0 to 99).

08 Functions: write once, run anywhere

```
function welcome (firstName) { ... }  →  welcome("David");
```

keyword · name · parameter · the work it does · calling it runs it ("David" = argument)

Two shapes, same result

```
function toFahrenheit(celsius) {
  return celsius * 1.8 + 32;
}

const toFahrenheit = (celsius) => {
  return celsius * 1.8 + 32;
};

toFahrenheit(30); // 86
```

Defining a function doesn't run it. **Calling** it by name does.

Parameter	Name in the definition
Argument	Real value you pass in
Order	1st argument → 1st parameter
Missing?	Skipped ones are `undefined`

return does two jobs

1. Sends a value back, so `sum(10, 10)` stands for `20`.
2. Stops the function. No lines after it run.

Logging a call and seeing `undefined`? The function forgot to `return`.

09 Arrays: one variable, many values

```
let arr = [20, 30, 40, 50, 100];
```

20	30	40	50	100	200
0	1	2	3	4	push

```
arr[0];              // 20 (first)
arr[arr.length - 1]; // 100 (last)
arr.push(200);       // adds to the end
```

Same zero-based index as strings. Code runs top to bottom, so a `push` only shows up on the lines **after** it.

.filter() keeps some, drops the rest

```
arr.filter((element) => {
  return element < 50;
});
```

20 30 40 ~~50~~ ~~100~~ → new array [20, 30, 40]

.map() changes every value

```
dollars.map(element => element * 100);
// [1, 5, 10] → [100, 500, 1000]
```

Both return a **new** array and leave the original alone.

The one-line shortcut

```
arr.filter(element => element < 50);
```

One parameter? Drop its `( )`. Body is **one** returned line? Drop the `{ }` and `return`.

`.forEach()` BONUS	Runs code for each item
`.find()` BONUS	Returns the first match

The same filter, the long way

```
let goodGrades = [];
for (let i = 0; i < grades.length; i++) {
  if (grades[i] !== "fail") {
    goodGrades.push(grades[i]);
  }
}
```

Interviewers may ask for both. Swap the if-and-push for `push(arr[i] * 100)` and it's `map`.

10 Objects: label your values

arrays line values up in order; objects give each one a name

```
let users = [
  {
    username: "David",
    email: "david@fes.com",
    password: "test123",
    lessonsCompleted: [0, 1],
  },
  { username: "Matri", /* ... */ },
];
```

`{ }` holds **name: value** pairs. Miss a **comma** between them and you get an error.

Read it with a dot	Gives you
`user.username`	`"David"`
`user.username[0]`	`"D"`
`users[0]`	The 1st object
`users[0].email`	`"david@fes.com"`
`users[0].lessonsCompleted`	`[0, 1]`

A property can hold anything: a string, a number, even an array. Values keep their type, so array properties get every array method.

".map is not a function"? You're holding an object, not an array. Map over a property that IS an array.

11 The DOM: change the page with JavaScript

Code	What it does
`document.querySelector(".title")`	**Your go-to.** Finds by CSS selector: `#id` `.class` `h1`
`document.getElementById("title")`	Finds by id. No `#` needed
`el.innerHTML = "Frontend Simplified"`	Replaces the content. `+=` adds onto it
`el.style.fontSize = "28px"`	Changes CSS. Two-word properties go **camelCase**
`el.classList.toggle("dark-theme")`	Adds the class, next time removes it (dark mode!)
`<button onclick="toggleDark()">`	Runs your function on click. Exact name + `()`
`el.addEventListener("click", fn)`	Same idea, done in JS instead of HTML BONUS

Getting null? Your JS ran before the HTML loaded. Add `defer` : `<script src="index.js" defer></script>` (or put the script at the end of the body).

12 Promises with async / await

a value that arrives later, usually from a backend

```
async function main() {
  try {
    const response = await fetch(".../users/1");
    const data = await response.json();
    emailRef.innerHTML = data.email;
  } catch (e) {
    emailRef.innerHTML = e;  // show the error
  }
}
```

Two promises to open: `fetch` gets the raw response, `.json()` makes it readable. Practice API: **jsonplaceholder.typicode.com/users**

`await` unlocks a promise and hands back the value.

`await` only works inside an `async` function. The error says so.

`try` runs first. If the promise rejects, `catch` gets the error.

Inside `async`, lines run in order, top to bottom.

Making one (the backend side)

```
new Promise((resolve, reject) => {
  resolve("VIP"); // or reject(error)
});
```